mathematics

MDPI

Article

A Hybrid Method for Optimization Modulo Theory of Floating-Point Numbers

Xu Yang [1,*], Liantao Song [1], Jiaqiang Yao [1], Zhipan Li [2] and Fan Yu [3]

[1] College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China; songliantao@nudt.edu.cn (L.S.); jiaqiangyao@nudt.edu.cn (J.Y.)
[2] College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China; fightinglzp@hnu.edu.cn
[3] School of Traffic and Transport Engineering, Central South University, Changsha 410075, China; yufan9705@163.com
* Correspondence: xuyang369@nudt.edu.cn

Abstract

Optimization Modulo Theory (OMT) is an important extension of Satisfiability Modulo Theory (SMT), aiming to find models that satisfy constraints while optimizing objective functions. OMT naturally arises in program analysis and formal methods, making OMT solving an important and challenging task. OptiMathSAT is currently the state-of-the-art OMT solver supporting floating-point theories. It incrementally calls an SMT solver to prune the search space step by step until the optimal objective value is found. However, this frequent invocation of the SMT solver is highly inefficient. Therefore, this paper proposes a novel hybrid method consisting of two stages: coarse search and precise search. The coarse search stage quickly finds an approximate optimal model using an optimization search algorithm. The precise search stage then performs an SMT-based binary search starting from the approximate model to find the true optimal model (the OMT solution). Evidently, this hybrid approach reduces computational cost by rapidly shrinking the search space, thereby improving overall solving performance. We evaluated our method on benchmarks derived from SMT-LIB2 and real-world programs. Experimental results show that our method outperforms the current state-of-the-art in both effectiveness and efficiency.

Keywords: OMT; floating-point; search-based optimization; constraint optimization

MSC: 68W50; 68N30

1. Introduction

Optimization Modulo Theory (OMT) is an important extension of Satisfiability Modulo Theory (SMT), in which the goal is to find a model that satisfies a given constraint formula while optimizing a specified objective [1–4]. In general, an OMT problem can be viewed as a combination of constraint satisfaction and mathematical optimization, where a feasible solution must satisfy logical constraints while minimizing or maximizing a given objective function. OMT problems naturally arise in program analysis and verification, such as computing extrema of variables under constraints [5] or deriving more precise abstract domains in abstract interpretation-based analyses [6].

OMT has attracted considerable attention from the research community. Nadel and Rychin [7] proposed an OMT solving method for the theory of unsigned bit-vectors, which optimizes the objective function lexicographically, bit by bit, from the most significant to

Check for updates

Academic Editor: Hsien-Chung Wu

Received: 1 April 2026
Revised: 17 April 2026
Accepted: 17 April 2026
Published: 20 April 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

the least significant. Subsequently, Trentin and Sebastiani [8] extended this approach to signed bit-vectors to directly solve OMT problems. They also proposed a similar method for floating-point arithmetic, introducing the concept of *attractor* to guide the search toward the optimal objective value [9]. To the best of our knowledge, this remains the only work addressing OMT over floating-point numbers. However, these approaches essentially explore the floating-point domain in a bit-wise and incremental manner, lacking guidance from the numerical structure of floating-point values and often requiring a large number of SMT solver invocations.

This paper focuses on optimization modulo theory of floating-point numbers (OMT(FP)). Existing approaches typically determine each bit of the objective function sequentially from the most significant to the least significant by repeatedly invoking an SMT solver, ultimately obtaining an optimal model. Clearly, such a search strategy lacks prior guidance and explores the entire floating-point domain incrementally, which often requires a large number of SMT calls and incurs significant computational overhead. This difficulty stems from the discrete, non-uniform, and non-linear structure of the floating-point domain, which makes efficient optimization particularly challenging. To address this limitation, we propose a hybrid method for OMT(FP). This method combines heuristic search with SMT-based precise reasoning: it first efficiently explores the floating-point space to obtain a high-quality approximately optimal model, and then refines this model via SMT solving to determine the true optimal model.

Based on this insight, we propose FOMT-H, a hybrid approach for optimization modulo theories over floating-point arithmetic. The method consists of two stages: a coarse search phase and a precise search phase. In the coarse search phase, the OMT problem is transformed into a constrained optimization problem, where the objective function corresponds directly to the original OMT objective, and the constraint function measures the degree of violation of the OMT constraints. Using the novel Floating-Point Evolutionary Algorithm (FPEA) proposed in this work, this phase efficiently explores the search space and identifies an approximate optimal model for the OMT instance. The SMT solver is then invoked to verify whether this candidate model is indeed optimal. If the candidate is not optimal, the algorithm proceeds to the precise search phase. In the precise search phase, we employ our SMT-based search procedure to refine the candidate and compute the true optimal model. The hybrid architecture allows the coarse search phase to rapidly shrink the search space by locating a high-quality approximation, while the precise search phase operates within this significantly reduced space to determine the true optimum. We compared our method, FOMT-H, with the state-of-the-art OMT(FP) solver, OptiMathSAT [10], on benchmarks derived from SMT-LIB2 and real-world programs. The results show that FOMT-H is both more effective and more efficient than the current state-of-the-art.

This paper makes the following contributions:

- We propose a hybrid two-phase framework for OMT(FP), consisting of a coarse search phase and a precise search phase, which bridges heuristic optimization and SMT-based precise solving.
- We develop a modeling framework that transforms OMT(FP) problems into mathematical constrained optimization problems. Furthermore, we propose an FPEA algorithm tailored to floating-point characteristics to efficiently search for approximately optimal models.
- We design an SMT-based binary search algorithm that leverages approximate solutions to significantly prune the search space and accelerate convergence.

- Experimental results on SMT-LIB2 and real-world benchmarks show that FOMT-H generally outperforms the state-of-the-art OMT(FP) solver OptiMathSAT in both effectiveness and efficiency.

The rest of the paper is organized as follows: Section 2 presents preliminaries and motivation; Section 3 describes our approach; Section 4 reports the experimental evaluation; Section 6 reviews related works; Section 7 is the conclusion.

2. Preliminary and Motivation

2.1. Floating-Point Numbers

In computer systems, real numbers are represented approximately using floating-point numbers, following the IEEE 754 standard [11]. A floating-point number F consists of a sign bit s, an exponent e, and a mantissa M:

$$F = (-1)^s \times M \times 2^e. \tag{1}$$

Here, $s \in \{0, 1\}$ indicates the sign, and the mantissa is $M = 1.f_1 f_2 \ldots f_m$ with an implicit leading 1 followed by fractional bits f_i. The exponent is derived from the biased exponent e' as $e = e' - (2^{p-1} - 1)$, where p denotes the number of bits used to encode the exponent. IEEE 754 defines multiple precision formats. In single precision, $p = 8$ bits are used for the exponent (bias 127) and 23 bits for the mantissa; in double precision, $p = 11$ bits are used for the exponent (bias 1023) and 52 bits for the mantissa.

Table 1 summarizes the five categories of floating-point numbers: Zero, Normalized Number, Denormalized Number, Infinity, and Not a Number. e_{min} and e_{max} denote the minimum and maximum unbiased exponent, respectively. The special floating-point numbers that will be used in the following sections are:

- The largest positive normalized number: single precision: 0 11111110 11111111111111 111111111, where $s = 0$, $e = e_{\max}$, $M = 2 - 2^{-23}$, yielding $F_{max}^{(\text{single})} = (2 - 2^{-23}) \times 2^{127}$; double precision: similarly, $F_{max}^{(\text{double})} = (2 - 2^{-52}) \times 2^{1023}$.
- The smallest positive denormalized (subnormal) number: single precision: 0 00000000 00000000000000000000001, where $s = 0$, $e = e_{\min} - 1$, $M = 2^{-23}$, yielding $F_{min}^{(\text{single})} = 2^{-23} \times 2^{-126} = 2^{-149}$; double precision: similarly, $F_{min}^{(\text{double})} = 2^{-52} \times 2^{-1022} = 2^{-1074}$.
- Infinity is represented by an all-ones exponent field and an all-zero fraction field. For example, single precision: 0 11111111 00000000000000000000000.

Table 1. IEEE 754 [11] Floating-Point Number Categories and Representations.

	Exponent	Fraction	Represents
Zero	$e = e_{min} - 1$	$f = 0$	± 0
Normalized Number	$e_{min} \leq e \leq e_{max}$	$f \neq 0$	$\pm 1.f \times 2^e$
Denormalized Number	$e = e_{min} - 1$	$f \neq 0$	$\pm 0.f \times 2^e$
Infinity	$e = e_{max} + 1$	$f = 0$	$\pm\infty$
Not a Number	$e = e_{max} + 1$	$f \neq 0$	NaN

2.2. Optimization Modulo Theory of Floating-Point Numbers

Let $\mathbb{F}$ be the IEEE 754 floating-point domain, and let $\mathbb{V}$ denote a set of floating-point variables. We consider floating-point expressions defined by the grammar

$$\begin{aligned} E &:= c \mid v \mid E_1 \otimes E_2 \mid G(E_1, \ldots, E_k), \\ \Psi &:= \neg\Psi \mid \Psi_1 \wedge \Psi_2 \mid \Psi_1 \vee \Psi_2 \mid E_1 \bowtie E_2, \end{aligned} \tag{2}$$

where $c \in \mathbb{F}$ is a constant, $v \in \mathbb{V}$ is a variable, $\otimes \in \{+, -, \times, /\}$ represents standard IEEE 754 arithmetic operations, and G is a k-ary arithmetic function. Floating-point constraints are arbitrary boolean combinations of atomic predicates of the form $E_1 \bowtie E_2$, where $\bowtie \in \{<, \leq, >, \geq, =, \neq\}$.

By straightforwardly applying De Morgan's laws [12] and the theory of floating-point arithmetic, any constraint formula Ψ can be rewritten into a negation-free Conjunctive Normal Form (CNF) [13]:

$$\Phi \equiv \bigwedge_{i=1}^{I} \bigvee_{j=1}^{J_i} \left(E_{i,j} \bowtie_{i,j} E'_{i,j} \right), \tag{3}$$

where each disjunction represents a clause consisting of atomic constraints. I denotes the number of clauses, J_i is the number of literals in the i-th clause, and $E_{i,j} \bowtie_{i,j} E'_{i,j}$ represents the j-th atomic constraint (literal) in the i-th clause.

An optimization objective is given by a floating-point expression t over $\mathbb{V}$. The goal of optimization modulo theory of floating-point numbers is to find an assignment to all variables that satisfies the input constraints and optimizes the value of t under IEEE 754 semantics.

Definition 1 (OMT(FP) [8]). *Let Φ be a floating-point constraint formula as defined above, and let t be a floating-point expression serving as the optimization objective. An* Optimization Modulo Theory of Floating-Point Numbers *problem asks for an assignment $\alpha^\star : \mathbb{V} \to \mathbb{F}$ such that $\alpha^\star \models \Phi$, and the objective t is optimized over all feasible assignments:*

$$\alpha^\star(t) = \begin{cases} \min\{\alpha(t) \mid \alpha \models \Phi\}, & \text{for minimization,} \\ \max\{\alpha(t) \mid \alpha \models \Phi\}, & \text{for maximization.} \end{cases} \tag{4}$$

The assignment $\alpha^\star$ that attains the optimal value is called the optimal model, *and $\alpha^\star(t)$ is the* optimal value. *If no assignment satisfies Φ, the OMT(FP) problem is unsatisfiable.*

2.3. Constrained Optimization

Constrained optimization problems aim to find the best solution under a set of constraints. A constrained optimization problem consists of variables, constraints over these variables, and an objective function to optimize.

Definition 2 (COP [14]). *A* Constrained Optimization Problem *is a triple*

$$\mathcal{P} = (x, \mathcal{C}(x), \mathcal{F}(x)), \tag{5}$$

where x is a vector of variables, $\mathcal{C}(x)$ is a set of constraints, and $\mathcal{F}(x)$ is the objective function.

A solution $x^\star$ satisfies the constraints, $x^\star \models \mathcal{C}(x)$, and optimizes the objective:

$$x^\star = \begin{cases} \arg\min_x \{\mathcal{F}(x) \mid \mathcal{C}(x) \textit{ holds}\}, & \text{for minimization,} \\ \arg\max_x \{\mathcal{F}(x) \mid \mathcal{C}(x) \textit{ holds}\}, & \text{for maximization.} \end{cases} \tag{6}$$

If no assignment satisfies $\mathcal{C}(x)$, the problem is infeasible.

2.4. Motivation

An OMT(FP) formula consists of a constraint formula Φ and an optimization objective t. OptiMathSAT [10] is currently the state-of-the-art solver for OMT(FP). It drives t toward the desired optimal floating-point value, referred to as *attr*. Specifically, it repeatedly

invokes an incremental SMT solver to determine each bit of t sequentially from the most significant to the least significant.

Consider an OMT(FP) formula $\Phi = x \geq 2;\ t = x$, where x is a floating-point variable with an exponent width of 3 bits and a significand width of 5 bits. Table 2 illustrates the minimization process of t [9]. Here, *attr* denotes a dynamically determined floating-point value toward which t is expected to converge. An underscore indicates the bit that currently needs to be determined by the SMT solver. When determining the i-th bit, if the solver returns SAT, $t[i]$ can be set to $attr[i]$; otherwise, $t[i]$ must be $\neg attr[i]$. Repeating this process until the least significant bit is fixed leads to $t = 0\ 100\ 0000_2 = 2$. Therefore, the resulting model of this OMT(FP) problem is $[x \mapsto 2]$. A closer inspection reveals that this search process essentially performs a binary search over the number of floating-point values. For example, $t[0] = 1$ ($t[0] = 0$) corresponds to $t < 0$ ($t \geq 0$), and $t[1] = 1$ ($t[1] = 0$) corresponds to $t \geq 2$ ($t < 2$), and so on. Clearly, from the perspective of the numerical values, this procedure is not a strict binary search. However, in terms of representable floating-point values, it is effectively a binary search: the interval $[0, 2]$ contains the same number of floating-point numbers as the region above 2.

Table 2. SMT-Based Bit-wise Enumeration Process for Optimal Solution Search. *attr* denotes the target objective value. The underline marks the bit to be decided by the SMT solver.

Attractor (*attr*)	SMT Query	Result	Constraint Update
$\underline{1}\ 111\ 0000\ (-\infty)$	$\Phi \wedge t[0] = 1$	UNSAT	$\Phi \leftarrow \Phi \wedge t[0] = 0$
$0\ \underline{0}00\ 0000\ (0)$	$\Phi \wedge t[1] = 0$	UNSAT	$\Phi \leftarrow \Phi \wedge t[1] = 1$
$0\ 1\underline{0}0\ 0000\ (2)$	$\Phi \wedge t[2] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[2] = 0$
$0\ 10\underline{0}\ 0000\ (2)$	$\Phi \wedge t[3] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[3] = 0$
$0\ 100\ \underline{0}000\ (2)$	$\Phi \wedge t[4] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[4] = 0$
$0\ 100\ 0\underline{0}00\ (2)$	$\Phi \wedge t[5] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[5] = 0$
$0\ 100\ 00\underline{0}0\ (2)$	$\Phi \wedge t[6] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[6] = 0$
$0\ 100\ 000\underline{0}\ (2)$	$\Phi \wedge t[7] = 0$	SAT	$\Phi \leftarrow \Phi \wedge t[7] = 0$

However, the aforementioned procedure essentially performs a binary search over the entire floating-point domain. This observation motivates the idea of pre-shrinking the interval in which the optimal objective t may lie. Suppose we already know that $[x \mapsto 4]$ is a satisfiable assignment for Φ; then we can restrict the optimal t to the interval $(-\infty, 4]$. Since the binary representation of 4 is 0 101 0000, the bit $t[2] = 0$ is strictly determined and does not require an SMT query for verification. Therefore, providing a tighter initial interval for the optimal t can facilitate the subsequent search for the true optimum. Based on this idea, we propose FOMT-H, which first employs an efficient search algorithm to find an assignment α that satisfies Φ and yields an approximate optimum $\alpha'(t)$ for t within the floating-point domain. Starting from this approximate optimum, FOMT-H then systematically performs an SMT-based binary search to accurately determine the true optimal value of t under the given constraint Φ.

3. Approach

FOMT-H is a hybrid OMT (FP) approach comprising a coarse search phase and a precise search phase. In the first phase, heuristic search algorithms are employed to rapidly explore the feasible solution space and obtain an approximately optimal model. Optimality checking is used to verify the authenticity of the approximate optimal solution, while unboundedness checking determines whether the optimal solution is a boundary value of the floating-point domain. These two checks are efficient and capable of determining the correct result in advance, thereby avoiding entry into the time-consuming precise search phase. In the second phase, starting from the approximately optimal model, precise SMT

solving processes are combined to further verify and refine the search results, thereby determining the true optimal model.

Algorithm 1 presents the overall execution process of FOMT-H. The algorithm takes as input floating-point constraints Φ, an optimization objective t, and a flag $flag$ indicating the optimization direction (where $flag$ is 1 for maximization problems and 0 otherwise). The output R of the algorithm can be SAT or UNSAT; if R is SAT, it indicates that an optimal model $\alpha^\star$ has been found. Note that in certain cases, the underlying SMT solver may crash, resulting in R being UNKNOWN, but this paper will not consider such scenarios.

Algorithm 1 FOMT-H

Input: Φ, t, $flag$, $Timeout$.
Output: R (SAT/UNSAT), $\alpha^\star$ (optimal model).
1: $\alpha' \leftarrow \text{CoarseSearch}(\Phi, t, \ldots)$ ▷ Algorithm 2
2: $ModelCheck(\Phi, \alpha')$ ▷ When α' violates Φ, call line 16
3: When $flag = 1$ then $\Phi \leftarrow \Phi \wedge t > \alpha'(t)$, otherwise $\Phi \leftarrow \Phi \wedge t < \alpha'(t)$
4: **if** Φ is unsatisfiable **then** ▷ Call the SMT solver
5: $R \leftarrow$ SAT
6: $\alpha^\star \leftarrow \alpha'$
7: **end if**
8: **if** Φ is satisfiable **then** ▷ Call the SMT solver
9: $Update(\alpha')$
10: When $flag = 1$ then $\Phi \leftarrow \Phi \wedge t = \infty$, otherwise $\Phi \leftarrow \Phi \wedge t = -\infty$
11: **if** Φ is satisfiable **then** ▷ Call the SMT solver
12: $R \leftarrow$ SAT
13: $\alpha^\star \leftarrow Update(\alpha')$
14: **end if**
15: **if** Φ is unsatisfiable **then** ▷ Call the SMT solver
16: $R, \alpha^\star \leftarrow \text{PreciseSearch}(\Phi, t, \ldots)$ ▷ Algorithm 3
17: **end if**
18: **end if**
19: **return** R, $\alpha^\star$

Algorithm 2 FPEA

Input: Φ, t, λ_1, λ_2, $seed$, $flag$, N, $Timeout$.
Output: α' (approximate optimal model).
1: $conv_1, conv_2 \leftarrow 0, 0$ ▷ $conv_1$ and $conv_2$ record the convergence of $\mathcal{C}'$ and $\mathcal{F}'$
2: $\mathcal{C}', \mathcal{F}' \leftarrow \infty, -\infty(\infty)$ ▷ If $flag$ is 1, then $\mathcal{F}'$ is $-\infty$, otherwise, it is ∞
3: $\mathbb{P} \leftarrow InitPopulaton(seed, \Phi, t)$ ▷ The size of $\mathbb{P}$ is N
4: $\mathbb{C} \leftarrow CalConsFunction(\mathbb{P}, \Phi)$ ▷ $\mathbb{C}$ is a set of $\mathcal{C}^\Phi$
5: $\mathbb{F} \leftarrow CalObjFunction(\mathbb{P}, t)$ ▷ $\mathbb{F}$ is a set of $\mathcal{F}^t$
6: **while** $conv_1 \leq \lambda_1$ **and** $conv_2 \leq \lambda_2$ **do** ▷ λ_1, λ_2 are the convergence thresholds
7: $cons_iter, obj_iter \leftarrow 0, 0$
8: **for each** $\mathcal{C}^\Phi(\alpha), \mathcal{F}^t(\alpha)$ **in** $\mathbb{C}, \mathbb{F}$ **do**
9: **if** $\mathcal{C}^\Phi(\alpha) < \mathcal{C}'$ **then** ▷ $\mathcal{C}'$ is the current minimum constraint violation
10: $cons_iter \leftarrow cons_iter + 1$
11: $\alpha', \mathcal{C}', \mathcal{F}' \leftarrow \alpha, \mathcal{C}^\Phi(\alpha), \mathcal{F}^t(\alpha)$ ▷ $\mathcal{F}'$ is the current optimal objective value
12: **end if**
13: **if** $\mathcal{C}^\Phi(\alpha) = \mathcal{C}'$ **and** $\mathcal{F}^t(\alpha) \prec \mathcal{F}'$ **then** ▷ $\prec$ represents a dominance
14: $obj_iter \leftarrow obj_iter + 1$
15: $\alpha', \mathcal{C}', \mathcal{F}' \leftarrow \alpha, \mathcal{C}^\Phi(\alpha), \mathcal{F}^t(\alpha)$
16: **end if**
17: **end for**
18: **if** $cons_iter = 0$ **then** ▷ No smaller $\mathcal{C}'$ exists
19: $conv_1 \leftarrow conv_1 + 1$
20: **end if**
21: **if** $obj_iter = 0$ **then** ▷ No better $\mathcal{F}'$ exists
22: $conv_2 \leftarrow conv_2 + 1$
23: **end if**
24: $\mathbb{P}' \leftarrow CoMutation(\mathbb{P})$ ▷ From the mutation operator pool
25: $\mathbb{C} \leftarrow CalConsFunction(\mathbb{P}', \Phi)$
26: $\mathbb{F} \leftarrow CalObjFunction(\mathbb{P}', t)$
27: $\mathbb{R} \leftarrow \mathbb{P} \cup \mathbb{P}'$ ▷ Merge parent and offspring
28: $\mathbb{L} \leftarrow ConsSort(\mathbb{R})$ ▷ Sorting by $\mathcal{C}^\Phi$
29: $\mathbb{L}' \leftarrow ObjSort(\mathbb{L})$ ▷ If $\mathcal{C}^\Phi$ is equal, sorting by $\mathcal{F}^t$
30: $\mathbb{P} \leftarrow SelectPop(\mathbb{L}')$ ▷ Update parent
31: **end while**

Algorithm 3 FOMT-BS

Input: Φ, t, $flag$, low (lower bound), $high$ (upper bound).
Output: R (SAT/UNSAT), $\alpha^\star$ (optimal model).

1: $R \leftarrow$ UNSAT
2: **while** $high - low > F^\star_{min}$ **do** ▷ $F^\star_{min}$ is the minimum floating-point number
3: $mid \leftarrow \frac{low}{2} + \frac{high}{2}$
4: When $flag = 1$ then $\Phi \leftarrow \Phi \wedge t \geq mid$, otherwise $\Phi \leftarrow \Phi \wedge t \leq mid$
5: **if** Φ is satisfiable **then** ▷ Call the SMT solver
6: $R \leftarrow$ SAT
7: $\alpha^\star \leftarrow Update(\alpha')$
8: When $flag = 1$ then $low \leftarrow \alpha'(t)$, otherwise $high \leftarrow \alpha'(t)$
9: **end if**
10: **if** Φ is unsatisfiable **then** ▷ Call the SMT solver
11: When $flag = 1$ then $high \leftarrow mid - F^\star_{min}$, otherwise $low \leftarrow mid + F^\star_{min}$
12: **end if**
13: **end while**
14: **return** R, $\alpha^\star$

Line 1 executes the coarse search process, invoking a heuristic search algorithm to obtain an approximately optimal model α'. The model α' returned by the search algorithm may fall into two cases: it either violates the constraints Φ or satisfies them. Line 2 detects the feasibility of α'; if α' violates the constraints, the algorithm directly proceeds to the precise search phase and uses $\alpha'(t)$ as the initial splitting point for the binary search algorithm. If α' satisfies the constraints, it still needs to be verified whether it is optimal. In the optimality check, we employ an SMT solver to verify whether α' is indeed the optimal solution. If it is optimal, the time-consuming precise search phase can be skipped.

Line 3 constructs optimality checking constraints for maximization and minimization problems, respectively, and submits them to the SMT solver. If the solver returns UNSAT, it indicates that α' is the true optimal model $\alpha^\star$, and the optimal objective function value $\alpha^\star(t)$ equals $\alpha'(t)$. The algorithm can terminate early and output R and $\alpha^\star$.

If the solver returns SAT, it indicates that a better model exists. At this point, we update α' to the model returned by the solver (see Line 9). Subsequently, FOMT-H performs an unboundedness check to detect whether the objective function t has reached the boundary value of floating-point numbers. Line 10 constructs unboundedness checking constraints for maximization and minimization problems, respectively, and submits them to the SMT solver.

If the solver returns SAT, it indicates that the optimal objective value $\alpha^\star(t)$ is ∞ (for maximization problems) or $-\infty$ (for minimization problems), and the algorithm can terminate immediately. Meanwhile, it outputs the result R and updates $\alpha^\star$ to the model currently returned by the solver.

If the solver returns UNSAT, it indicates that the true optimal value $\alpha^\star(t)$ satisfies $\alpha^\star(t) \in [\alpha'(t), F_{max}]$ (for maximization problems) or $\alpha^\star(t) \in [-F_{max}, \alpha'(t)]$ (for minimization problems), where F_{max} is the largest representable floating-point number. Therefore, the algorithm proceeds to the precise search phase (see Line 16).

3.1. Coarse Search

In the coarse search stage, we first relax the OMT(FP) problem into a traditional mathematical COP, and then employ FPEA, which is specifically designed for floating-point characteristics, to solve this problem and obtain an approximately optimal model. This strategy utilizes a heuristic search method to explore possible feasible solutions in the vast floating-point space, thereby narrowing the search space for the subsequent precise search process and improving the overall optimization efficiency.

3.1.1. Problem Transformation

An OMT(FP) formula consists of a constraint component Φ and an optimization objective t. The following shows how Φ and t can be mathematically modeled as a COP:

$$\mathcal{P} = (\alpha, \mathcal{C}^{\Phi}(\alpha), \mathcal{F}^{t}(\alpha)). \tag{7}$$

Here, α denotes the current assignment, representing a concrete instantiation of the decision variables in the optimization problem, with the decision space defined over the entire floating-point domain. $\mathcal{F}^{t}(\alpha)$ represents the value of the objective t under the current assignment, i.e., $\mathcal{F}^{t}(\alpha) = \alpha(t)$. $\mathcal{C}(\alpha)$ denotes the degree of constraint violation of Φ under the assignment α.

The degree of constraint violation is defined as follows:

$$\mathcal{C}^{\Phi}(\alpha) := \sum_{i=1}^{I} \min_{1 \le j \le J_i} \mathcal{D}(\alpha, E_{i,j} \bowtie_{i,j} E'_{i,j}). \tag{8}$$

Consider the conjunctive normal form in Section 2.2 (i.e., Formula (3)). For constraints connected by $\bigvee$, the minimum violation degree among the atomic constraints is selected as the violation degree of the constraint, since the satisfaction of any disjunct makes the entire clause satisfied. For constraints connected by $\bigwedge$, the violation degrees of all constraints are summed, since the entire conjunction is satisfied only when all conjuncts are satisfied. $\mathcal{D}$ denotes the violation degree of an atomic constraint under a given assignment.

The degree of violation of atomic constraints $\mathcal{D}$ is calculated as follows:

$$\mathcal{D}(\alpha, E_1 \bowtie E_2) := \begin{cases} 0 & \text{if } \alpha(E_1 \bowtie E_2) \text{ is satisfied,} \\ 1 & \bowtie \in \{\neq\}, \\ \frac{\mathcal{H}(Bin_{\alpha(E_1)}, Bin_{\alpha(E_2)})}{\mathcal{M}} & \bowtie \in \{=\}, \\ \frac{|Int_{\alpha(E_1)} - Int_{\alpha(E_2)}| + \epsilon}{2^{\mathcal{M}} - 1} & \bowtie \in \{\le, \ge, <, >\}. \end{cases} \tag{9}$$

where $\alpha(E)$ evaluates expression E under assignment α. $\mathcal{H}$ denotes the Hamming distance [15], i.e., the number of bit positions at which two binary representations differ. Bin_x is the binary representation of the floating-point number x, Int_x is the integer representation of the floating-point number x, and $\mathcal{M}$ is the bit width of the floating-point number, and $\epsilon = 1$ for $\{<, >\}$ else 0. When $x \ge 0$, Int_x is the integer representation of Bin_x. When $x < 0$, Int_x is the integer representation of $2^{\mathcal{M}-1} - Bin_x$.

In Equation (9), if $\alpha(E_1 \bowtie E_2)$ evaluates to true, i.e., the constraint is satisfied, then the value of $\mathcal{D}$ is 0; otherwise, $\mathcal{D}$ is computed according to the corresponding violation function. For $\bowtie \in \{\neq\}$, the value of $\mathcal{D}$ is always 1. For $\bowtie \in \{=\}$, the Hamming distance is used to quantify the difference between two floating-point numbers. For example, let x and y be 32-bit floating-point numbers with $x = 2.0$ and $y = 36893488147419103000.0$. Their binary representations are Bin_x = 0 10000000 00000000000000000000000 and Bin_y = 0 11000000 00000000000000000000000. The Hamming distance between x and y is 1, whereas their numerical difference is as large as 36893488147419102998. This example shows that, compared to directly using the Euclidean distance [16] between floating-point values, the Hamming distance better captures bit-level similarity.

For relational predicates $\bowtie \in \{\le, \ge, <, >\}$, the Euclidean distance is computed on the corresponding integer representations rather than the floating-point values. This is because, in the IEEE 754 standard, the distribution of floating-point numbers across different intervals is non-uniform: larger floating-point numbers are more sparsely represented than

smaller ones. In other words, the distance between two adjacent large floating-point numbers is greater than that between two adjacent small ones. In addition, computing the Euclidean distance based on the corresponding integer representation can avoid underflow during normalization. For example, consider two 32-bit floating-point numbers $x = 1.0$ and $y = 2.0$. According to the IEEE 754 standard, $\frac{|x-y|}{F_{max}}$ may underflow to a subnormal value. In contrast, let $Int_x = 1073741824$ and $Int_y = 1065353216$, then $\frac{|Int_x - Int_y|}{2^{\mathcal{M}}-1} \approx 0.00195$. Since $\frac{1}{2^{\mathcal{M}}-1}$ is much larger than $F_{\min}$, using integer representations instead of floating-point values effectively avoids underflow in the normalization process.

Finally, all constraint violation degrees are normalized to the interval $[0, 1]$. The reason for this is to map the violation degrees of atomic constraints into the range $[0, 1]$, thereby balancing the influence of different predicate atomic constraints on the overall constraint violation degree. For example, for the $\neq$ predicate, the violation degree can only take the value 1, whereas for the $=$ predicate, the violation degree may become arbitrarily large. Without normalization, the search process may focus excessively on the $=$ atomic constraints while neglecting the $\neq$ atomic constraints.

An Example

Assume that Φ is $x \leq 64.0 \wedge y = 64.0$, and the objective variable t is equal to x and is to be maximized, where x and y are 32-bit floating-point numbers. Given an assignment $\alpha = [x \mapsto 63.0, y \mapsto 64.0]$, we have $\mathcal{F}^t(\alpha) = 63.0$. The constraint violation $\mathcal{C}^{\Phi}(\alpha)$ is computed according to Formulas (8) and (9) as follows.

$$\mathcal{C}^{\Phi}(\alpha) = \frac{1}{2} \times \Big(\mathcal{D}(\alpha, x \leq 64.0) + \mathcal{D}(\alpha, y = 64.0) \Big) = \frac{1}{2} \times (0 + 0) = 0. \tag{10}$$

Clearly, α is a model that satisfies Φ, but it is not an optimal model. In this simple example, it is easy to observe that the optimal model is $\alpha^{\star} = [x \mapsto 64.0, y \mapsto 64.0]$.

3.1.2. Optimization Solving

The floating-point domain in modern machines forms a discrete and highly irregular search space, and the function $\mathcal{D}$ is both discontinuous and non-differentiable. Therefore, we employ an evolutionary algorithm [17,18] as the optimization engine, leveraging its strong global search capability and robustness in non-smooth landscapes. Evolutionary algorithms are bio-inspired optimization techniques that mimic the process of natural evolution. They operate on two fundamental concepts: An *Individual* and *Population* [17]. To more effectively search within the floating-point space, we design a specialized evolutionary algorithm tailored to the characteristics of OMT(FP), referred to as the Floating-Point Evolutionary Algorithm (FPEA). Individual is defined as a triple $(X, \mathcal{C}, \mathcal{F})$, where $X = \{\alpha(v_1), \alpha(v_2), \ldots, \alpha(v_n)\}$ is the decision vector consisting of the floating-point assignments to variables v appearing in Φ, and n denotes the number of such variables. The value $\mathcal{C}$ represents the constraint-violation score of X, and $\mathcal{F}$ denotes the corresponding objective value. Population is then defined as a set of such individuals.

Algorithm 2 presents the workflow of the FPEA procedure. The inputs are as follows: Φ is the constraint part of the OMT(FP) problem; t is the optimization objective; *seed* is a set of constants extracted from Φ, and this extraction is performed concurrently with the problem transformation process; λ_1 and λ_2 are hyperparameters controlling the termination conditions, representing the maximum number of generations allowed without producing improvements in $\mathcal{C}'$ and $\mathcal{F}'$, respectively; N denotes the population size, which is set empirically; *Timeout* specifies the time limit; $flag$ indicates the optimization direction, where 1 for maximization and 0 for minimization. The outputs an approximate optimal model α'. The execution process of FPEA can be divided into the following six stages:

Initialization

Lines 1-2 set two global parameters $conv_1$ and $conv_2$ to record the convergence of $\mathcal{C}'$ and $\mathcal{F}'$, indicating the convergence status of the algorithm. Specifically, these two global parameters $\mathcal{C}'$ and $\mathcal{F}'$ represent the current minimum constraint violation and the optimal objective function value of the algorithm, respectively. Line 2 initializes $\mathcal{C}'$ to ∞, and initializes $\mathcal{F}'$ to $-\infty$ (for maximization problems) or ∞ (for minimization problems). Line 3 performs population initialization using the preprocessing *seed*, generating a population $\mathbb{P}$ containing N individuals. Specifically, for each $s \in seed$, an individual is created with its decision vector $X = (s, \ldots, s)$, while the remaining individuals are generated randomly.

Evaluation

Lines 4–5 compute the constraint violation and objective value for every individual in the population $\mathbb{P}$. Lines 7–17 check whether the current population contains a dominant individual. An individual is considered dominant if its constraint violation is smaller than $\mathcal{C}'$, or if its constraint violation equals $\mathcal{C}'$ and its objective value is better than $\mathcal{F}'$. Note that when the OMT(FP) problem is a maximization task, $\prec$ corresponds to $>$; otherwise, for minimization, $\prec$ corresponds to $<$. Lines 18–23 count the number of generations in which no superior individual is produced, denoted as $conv_1$ and $conv_2$. Subsequently, Line 6 checks whether $conv_1$ and $conv_2$ exceed the thresholds λ_1 and λ_2. The algorithm terminates if no superior individual is generated within the specified number of iterations or if a timeout occurs.

Mutation

Line 24 adopts a cooperative variation strategy, which randomly selects a mutation operator from an operator pool to mutate each individual in the parent population $\mathbb{P}$. Notably, all crossover and recombination operations are performed on the corresponding X of each individual. The operator pool contains nine crossover and mutation operators, including Single-point crossover [19], Uniform crossover [20], Frame-shift mutation [21], Reverse mutation [22], Single-point mutation [23], Single-point byte mutation [23], Exponent-byte mutation [23], Special-number mutation [23], and Objective mutation. **Objective mutation** is specifically designed for OMT(FP). It forces the optimization objective to mutate only toward ∞ (for maximization) or $-\infty$ (for minimization). This directed mutation strategy helps accelerate the search toward better approximate solutions. To evaluate the quality of the offspring, Lines 25–26 compute $\mathcal{C}^{\Phi}$ and $\mathcal{F}^t$ for every individual in the offspring population $\mathbb{P}'$.

Selection

Line 27 merges the parent and offspring populations into $\mathbb{R}$. Line 28 sorts the population $\mathbb{R}$ in ascending order of constraint violation $\mathcal{C}^{\Phi}$. Based on this sorting, the population is further ordered by the objective function values (as in Line 29). For maximization problems, the population $\mathbb{L}$ is sorted in descending order, whereas for minimization problems, it is sorted in ascending order. The merged population $\mathbb{L}'$ establishes a complete partial order. Line 30 performs the selection operation: the top N individuals from the $2 \times N$ population $\mathbb{L}'$ are chosen as the parent population $\mathbb{P}$ for the next iteration. Finally, when the algorithm satisfies the termination conditions or reaches the timeout, it outputs an approximate optimal model α'. Note that since the termination condition does not require $\mathcal{C}^{\Phi} = 0$, the model α' may or may not satisfy the formula Φ.

3.2. Precise Search

If the model α' returned by Algorithm 2 does not satisfy Φ, the binary search in this stage still needs to be conducted over the entire floating-point value range. If α' satisfies Φ, and neither the approximate optimality check nor the unboundedness check can determine the true optimal model $\alpha^\star$, then $\alpha'(t)$ is used as the lower bound low (for maximization problems) or the upper bound $high$ (for minimization problems) for the binary search. Algorithm 3 describes the procedure of FOMT-BS, whose basic idea is SMT-based binary search. The inputs include Φ, t, the lower bound low and upper bound $high$ of the target value, and the output is SAT or UNSAT. If SAT is returned, the true optimal model $\alpha^\star$ is also provided. Line 1 must initialize R to UNSAT to ensure that when the entire solution space has been traversed, if no model satisfying Φ is found, the algorithm ultimately returns UNSAT. Line 2 is the termination condition, where $F^\star_{min}$ is a floating-point number smaller than F_{min}, called the smallest subnormal floating-point number, represented as $0.0, ..., 1 \times 2^{e_{min}}$. Line 3 does not directly compute $\frac{low+high}{2}$ to avoid overflow and floating-point errors. Line 4 checks whether there exists a value better than the current $\alpha'(t)$ in the interval $[mid, high]$ (for maximization problems) or $[low, mid]$ (for minimization problems). If a better solution exists, α' is updated to the current optimal model, $\alpha^\star$ is updated to α' (see Line 7), and R is also marked as SAT (see Line 6). Line 8 updates low (for maximization problems) or $high$ (for minimization problems) to $\alpha'(t)$ to accelerate interval contraction and improve the efficiency of binary search. If the SMT solver returns UNSAT, then $high$ (for maximization problems) or low (for minimization problems) will be updated to $mid - F^\star_{min}$ (for maximization problems) or $mid + F^\star_{min}$ (for minimization problems) (see Line 11). It should be noted that $\alpha^\star$ is generated only when R is SAT, to ensure that the output $\alpha^\star$ satisfies the formula Φ; otherwise, $\alpha^\star$ is empty. The termination condition in Line 2 indicates that the SMT-based binary search process terminates when there is no representable floating-point number between low and $high$.

3.3. Soundness and Completeness

Consider an OMT(FP) problem consisting of a constraint formula Φ and an optimization objective t. An assignment $\alpha^\star$ is a solution (optimal model) of this problem if and only if it satisfies Φ and optimizes t.

Definition 3 (Soundness of FOMT-H)**.** *If FOMT-H returns SAT with a model α, then α must be the optimal model $\alpha^\star$ of the OMT(FP) problem. If FOMT-H returns UNSAT, then the OMT(FP) problem has no solution.*

Definition 4 (Completeness of FOMT-H)**.** *If the OMT(FP) problem has a solution, then FOMT-H returns SAT and obtains the optimal model $\alpha^\star$. If the OMT(FP) problem has no solution, then FOMT-H returns UNSAT.*

Theorem 1 (Soundness and Completeness of FOMT-H)**.** *FOMT-H is both sound and complete for solving OMT(FP) problems.*

Proof. When FOMT-H returns SAT, the model α' must be produced either during the optimality check (line 6 of Algorithm 1), the unboundedness check (line 13 of Algorithm 1), or the precise search (line 7 of Algorithm 3). For the optimality check, when the solver returns UNSAT, it indicates that no better model exists; hence, α' is $\alpha^\star$. For the unboundedness check, when the solver returns SAT and produces a model α' such that $\alpha(t) = \infty$ (or $-\infty$), α' is therefore $\alpha^\star$. For the precise search, when the solver returns SAT, α' necessarily satisfies Φ. The termination condition $high - low \leq \mathcal{F}^\star_{min}$ guarantees that there is no representable

floating-point value between low and $high$. Since α' is continuously updated as the current best model, the final α' is $\alpha^\star$.

When FOMT-H returns UNSAT, this must occur in the precise search phase. Since the SMT-based binary search exhausts the entire floating-point domain without finding an assignment satisfying Φ, the OMT(FP) problem has no solution.

Assume that the OMT(FP) problem has a solution. In the best case, $\alpha^\star$ is determined during the optimality or unboundedness checks, and FOMT-H returns SAT. In the worst case, FOMT-H enters the precise search phase, where FOMT-BS performs an SMT-based binary search over the interval $[low, high]$. Since the interval contains only finitely many floating-point values, the search must eventually find the true optimal solution $\alpha^\star$, after which FOMT-H returns SAT.

Assume that the OMT(FP) problem has no solution. Both the approximate optimality check and the unboundedness check fail, and FOMT-H directly enters the precise search phase. The binary search explores all floating-point values in the interval $[low, high]$ until $high - low \leq \mathcal{F}^\star_{min}$. If R remains unchanged when FOMT-BS terminates, FOMT-H returns UNSAT. □

Note that the above proof assumes that the SMT solver is sound and complete, and excludes cases where it returns UNKNOWN due to internal failures. The FPEA algorithm may fail to find a feasible solution, but this only affects the efficiency of FOMT and does not compromise the correctness of the final result. In theory, FOMT-BS will return the correct result given sufficient time; therefore, UNKNOWN results caused by timeouts do not affect the validity of the proof.

4. Implementation and Evaluation

4.1. Implementation

FOMT-H is a prototype adopting a hybrid strategy, implemented based on QSF [23] and multiple SMT solvers (e.g., MathSAT5 (v5.6.11) [24] and Bitwuzla (v0.8.2) [25]), while also relying on Z3 (v4.13.0) [26], LLVM (v6.0.0) [27], and NLopt (v2.7.1) [28]. The input of FOMT-H is a file in SMT-LIB2 format [29]. Z3 serves as the front-end parser to process the input formulas; LLVM is responsible for generating code for the constraint and objective functions; and FPEA is built on top of NLopt as the optimization engine for the transformed constraint optimization problem. We also implemented a prototype that adopts only the precise search strategy, named FOMT-BS, which relies on the underlying SMT solver and Z3 (used for parsing SMT-LIB2 format). For simplicity, we use FOMT as a collective term to refer to both FOMT-H and FOMT-BS in the remainder of this paper.

4.2. Research Questions

We aim to answer the following research questions (RQs):

- **RQ1**: How does FOMT perform in terms of effectiveness and efficiency compared to state-of-the-art methods?
- **RQ2**: What impact do different SMT solvers have on the effectiveness and efficiency of FOMT?
- **RQ3**: Can Hybrid strategies improve the overall performance of FOMT-H?

4.3. Experimental Setup

4.3.1. Benchmarks

We evaluated FOMT-H on both the QF_FP benchmark and the real-world program benchmark. The QF_FP benchmark is derived from the standard QF_FP SMT-LIB2 benchmarks [29]. We only consider satisfiable instances in QF_FP, and for each variable in a

formula, we generate two QF_FP OMT instances: one for maximization and one for minimization. If the original QF_FP SMT formula has only a single satisfiable solution, some constraints are appropriately relaxed or removed. In total, we automatically generated 29,298 QF_FP OMT instances. These instances fall into four categories: `griggio`, `heizmann`, `schanda`, and `wintersteiger`. From each category, we randomly selected 300 instances (or all instances if fewer than 300), resulting in a benchmark containing 677 instances. This benchmark is also used in the experimental evaluation of the OptiMathSAT paper [10].

The real-world program (RWP) benchmark is constructed by collecting path constraints from the symbolic execution of floating-point programs. We used the symbolic execution tool KLEE [30] to analyze floating-point programs in the GNU Scientific Library (GSL) [31], obtaining 3493 path-constraint instances [32]. Similarly, using an OMT formula transformation script, we obtained 3493 instances for the experiments in this section, forming this benchmark.

4.3.2. Baseline

To the best of our knowledge, OptiMathSAT is currently the only publicly available OMT solver that supports floating-point theory. Therefore, we use it as the primary baseline in our experimental evaluation. FOMT-H is the floating-point constraint optimization approach proposed in this chapter, whose overall framework relies on the capability of underlying SMT solvers. Since OptiMathSAT is built on top of MathSAT5, to ensure a fair comparison, we implemented FOMT-H-MathSAT5 (abbreviated as FOMT-H-M) based on MathSAT5. Furthermore, to evaluate the impact of using Bitwuzla, the winner of the SMT-COMP QF_FP track, on the performance of FOMT-H, we also implemented FOMT-H-Bitwuzla (abbreviated as FOMT-H-B). Finally, to analyze the contribution of the coarse search phase in FOMT-H to the overall performance, we implemented variants that retain only the precise search phase, namely FOMT-BS-MathSAT5 (abbreviated as FOMT-BS-M) and FOMT-BS-Bitwuzla (abbreviated as FOMT-BS-B).

4.3.3. Configuration

All experiments were conducted on a server equipped with an Intel(R) Xeon(R) Gold 6458Q 128-core CPU @ 3.10GHz and 512 GB of memory. The timeout for all methods was uniformly set to 600 s. OptiMathSAT was executed with its default configuration, and the SMT solvers used in FOMT-H were also set to their default configurations. For the proposed FPEA algorithm, the population size was set to 100, the timeout was set to 5 s, and both λ_1 and λ_2 were set to 1000. All experiments were repeated ten times to improve the reliability of the results.

4.4. FOMT vs. OptiMathSAT

Table 3 compares the numbers of solved instances by FOMT-H-MathSAT5, FOMT-BS-MathSAT5, and OptiMathSAT on the QF_FP benchmark and the real-world program (RWP) benchmark. The value outside the parentheses denotes the number of successfully solved instances, while the percentage inside the parentheses indicates the proportion with respect to the total number of instances. For each instance, each method was executed independently ten times. An instance is considered successfully solved by a method if the number of successful runs exceeds the number of failed runs; otherwise, it is regarded as unsolved. **Both** denotes the number of instances successfully solved by both methods, **Only OptiMathSAT** denotes the number of instances solved only by OptiMathSAT, **Only FOMT** denotes the number of instances solved only by FOMT, and **Neither** denotes the number of instances unsolved by both methods.

Table 3. Comparison of instance counts between OptiMathSAT and FOMT-MathSAT5 on two benchmarks. The **Strategy** column denotes the hybrid strategy and the precise-search-only strategy, respectively.

Benchmark	Strategy	Both	Only OptiMathSAT	Only FOMT	Neither
QF_FP	FOMT-H-M	552 (≈82.76%)	4 (≈0.60%)	42 (≈6.30%)	69 (≈10.34%)
	FOMT-BS-M	547 (≈85.91%)	9 (≈0.15%)	35 (≈1.50%)	76 (≈12.44%)
RWP	FOMT-H-M	2939 (≈84.14%)	12 (≈0.35%)	435 (≈12.45%)	107 (≈3.06%)
	FOMT-BS-M	2774 (≈79.42%)	89 (≈2.55%)	177 (≈5.07%)	453 (≈12.96%)

On the QF_FP benchmark, FOMT-H-M and FOMT-BS-M successfully solved 594 and 582 instances, respectively, while OptiMathSAT solved 556 instances. These results indicate that both FOMT variants outperform OptiMathSAT in terms of both the total number of solved instances and the number of uniquely solved instances. On the real-world program benchmark, FOMT-H-M and FOMT-BS-M successfully solved 3374 and 2951 instances, respectively, while OptiMathSAT solved 2951 instances. FOMT-H-M achieves the best overall solving capability on this benchmark and again shows advantages in both the total number of solved instances and the number of uniquely solved instances.

Moreover, in a small number of instances, OptiMathSAT succeeds while FOMT-H-M fails. This is mainly because the solution space is too small, causing the coarse search phase to fail and waste computational time. There are also cases where OptiMathSAT succeeds but FOMT-BS-M fails, since the optimal solutions of these instances are likely located near boundary values and are therefore not well-suited for value-based binary search methods. The unsolved instances in the **Neither** category are mainly due to the inability to complete solving within the given time limit; that is, the failures are primarily caused by timeouts rather than incorrect decisions.

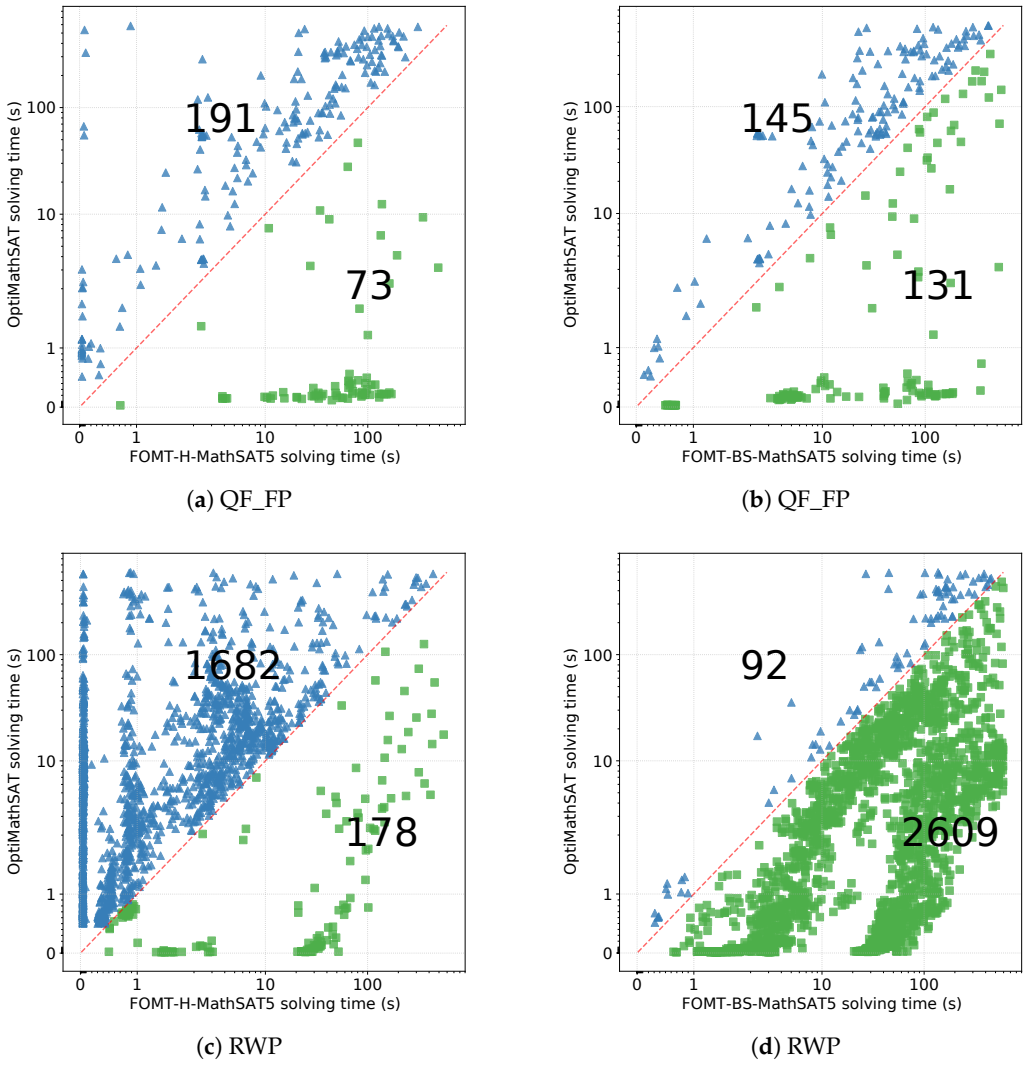
Table 4 presents the statistical significance analysis results for all methods. The table reports the Vargha–Delaney A^{12} effect size based on multiple runs, which represents the probability that method M_A outperforms method M_B in terms of the number of solved instances. It also lists the statistically significant winner determined by the Wilcoxon signed-rank test ($p < 0.001$). The results clearly show that FOMT-H-M and FOMT-BS-M significantly outperform OptiMathSAT on both the QF_FP benchmark and the real-world program benchmark.

Table 4. Statistical significance analysis of compared methods on the two benchmark sets, reporting Vargha–Delaney A^{12} effect sizes and p-values (Wilcoxon signed-rank test).

M_A vs. M_B		QF_FP			RWP		
		A^{12}	Winner	p-Value	A^{12}	Winner	p-Value
FOMT-H-M	OptiMathSAT	1	M_A	0.000	1	M_A	0.000
FOMT-BS-M	OptiMathSAT	1	M_A	0.000	1	M_A	0.000
FOMT-H-B	FOMT-H-M	1	M_A	0.000	1	M_A	0.000
FOMT-BS-B	FOMT-BS-M	1	M_A	0.000	1	M_A	0.000
FOMT-H-M	FOMT-BS-M	1	M_A	0.000	1	M_A	0.000

Figure 1 shows the comparison of solving time between FOMT-MathSAT5 and OptiMathSAT on the QF_FP benchmark and the real-world program (RWP) benchmark. In the scatter plots, the x-axis represents the execution time of FOMT-MathSAT5, while the y-axis represents the execution time of OptiMathSAT. Each point in the plot corresponds to a comparable instance (the confidence intervals do not overlap at the 99% confidence level), indicating instances that can be judged as significantly better or worse than the compared method. The numbers on the two sides of the diagonal line denote the counts of instances where the corresponding method wins (i.e., has shorter solving time), and the remaining

instances are incomparable (the confidence intervals overlap at the 99% confidence level). It is worth noting that we consider both methods to perform well on instances that can be solved in a very short time, as such instances are relatively simple and indicate high efficiency for both methods. Therefore, we filtered out instances with execution time less than 0.5 s.



(**a**) QF_FP (**b**) QF_FP

(**c**) RWP (**d**) RWP

Figure 1. Execution time comparison between FOMT-MathSAT5 and OptiMathSAT on the two benchmarks.

For the QF_FP benchmark, both FOMT-H-M and FOMT-BS-M are faster than OptiMathSAT on more instances, demonstrating a clear overall efficiency advantage. However, the hybrid strategy performs worse than OptiMathSAT on 73 instances, mainly due to failures in the coarse search phase. The strategy using only precise search performs worse than OptiMathSAT on 131 instances, indicating that these instances are better suited to bit-level enumeration search. In other words, whether using the hybrid two-phase strategy or the precise-search-only strategy, FOMT achieves better solving efficiency than OptiMathSAT on this benchmark.

On the real-world program benchmark, FOMT-H-M shows significantly shorter solving time than OptiMathSAT on more instances, whereas the overall solving efficiency of FOMT-BS-M is inferior to OptiMathSAT. This indicates that the hybrid strategy plays a crucial role in improving solving efficiency for real-world programs. The main reason is that, for some real-world instances, the optimal solutions often lie near the boundaries of

the floating-point domain. In such cases, the binary search strategy based on floating-point values typically requires many iterations to converge. In contrast, OptiMathSAT adopts a binary search strategy based on the number of floating-point values in the domain, which can locate the optimal solution with fewer iterations in such scenarios, thereby achieving higher solving efficiency. Overall, the coarse search phase of FOMT-H improves efficiency on most instances, though its effectiveness depends on the constraint formula.

Answer to **RQ1**: **Effectiveness.** On both benchmarks, under the same timeout limit, FOMT solves more instances than the state-of-the-art solver OptiMathSAT, using either the hybrid strategy or the precise-search-only strategy. **Efficiency.** On instances solved by both approaches, FOMT-H is significantly faster than OptiMathSAT on both benchmarks, while FOMT-BS only outperforms OptiMathSAT on the QF_FP benchmark.

4.5. FOMT-MathSAT5 vs. FOMT-Bitwuzla

Table 5 presents a performance comparison between FOMT-H-MathSAT5 and FOMT-H-Bitwuzla on the QF_FP benchmark and the real-world program (RWP) benchmark. The fields shown in the table are consistent with those in Table 3, except that the **Strategy** column is removed; descriptions of the remaining columns are omitted for brevity. In terms of solving scale, FOMT-H-M successfully solves 635 and 3399 instances on the QF_FP and RWP benchmarks, respectively, whereas FOMT-H-M solves 594 and 3374 instances on the same benchmarks. These results indicate that FOMT-H-M not only achieves a higher total number of solved instances, but also demonstrates stronger performance in terms of uniquely solved instances.

Table 5. Comparison of the number of solved instances between FOMT-H-M and FOMT-H-B on the two benchmarks. **Both** denotes the number of instances successfully solved by both methods, **Only FOMT-H-M** denotes the number of instances solved only by FOMT-H-M, **Only FOMT-H-B** denotes the number of instances solved only by FOMT-H-B, and **Neither** denotes the number of instances unsolved by both methods.

Benchmark	Both	Only FOMT-H-M	Only FOMT-H-B	Neither
QF_FP	587 (≈88.00%)	7 (≈1.05%)	48 (≈7.20%)	25 (≈3.75%)
RWP	3358 (≈96.14%)	16 (≈0.46%)	41 (≈1.17%)	78 (≈2.23%)

Table 6 presents the solving performance comparison between FOMT-BS-MathSAT5 and FOMT-BS-Bitwuzla on the QF_FP benchmark and the real-world program (RWP) benchmark. It reports the number of solved instances for both approaches on each benchmark. All metrics in the table are consistent with those in Table 5, and their descriptions are omitted for brevity. According to the experimental results, FOMT-BS-B successfully solves 627 and 3031 instances on the QF_FP and RWP benchmarks, respectively, whereas FOMT-BS-M solves 582 and 2858 instances on the same benchmarks. These results show that FOMT-BS-B not only achieves better overall solving capability, but also outperforms in terms of the number of instances uniquely solved by the method, demonstrating stronger performance.

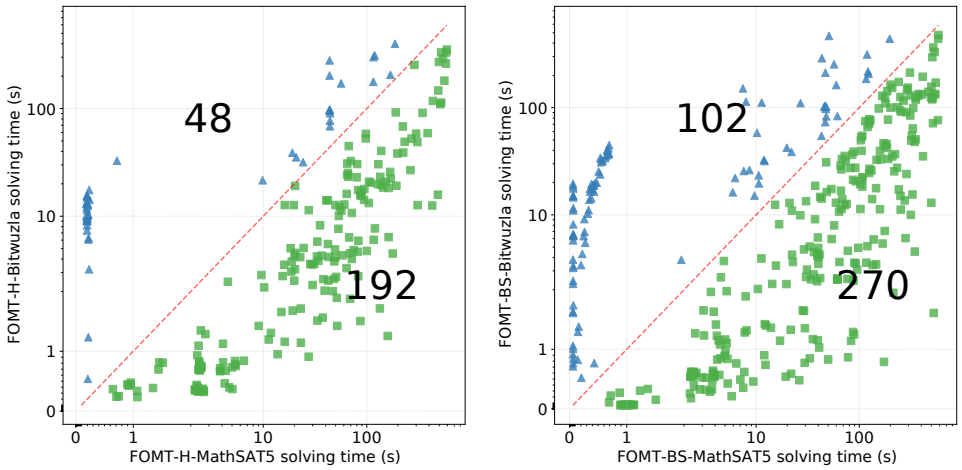
Table 6. Comparison of the number of solved instances between FOMT-BS-M and FOMT-BS-B on the two benchmarks. **Both** denotes the number of instances successfully solved by both methods, **Only FOMT-BS-M** denotes the number of instances solved only by FOMT-BS-M, **Only FOMT-BS-B** denotes the number of instances solved only by FOMT-BS-B, and **Neither** denotes the number of instances unsolved by both methods.

Benchmark	Both	Only FOMT-BS-M	Only FOMT-BS-B	Neither
QF_FP	575 (≈86.20%)	7 (≈1.05%)	52 (≈7.80%)	33 (≈4.95%)
RWP	2797 (≈80.07%)	66 (≈1.89%)	234 (≈6.70%)	396 (≈11.34%)

By jointly analyzing Tables 5 and 6, we observe that the Bitwuzla-based FOMT does not consistently outperform the MathSAT5-based FOMT; instead, the two exhibit complementary strengths. This is because different SMT solvers excel in different solving scenarios. Since FOMT adopts a search-based strategy, it is necessary to conduct statistical significance analysis. Table 4 reports the statistical significance results for all compared approaches. The analysis shows that the performance improvements of FOMT-H-B and FOMT-BS-B on both the QF_FP and RWP benchmarks are statistically significant, indicating that they provide stable and reliable advantages in solving capability compared with FOMT-H-M and FOMT-BS-M.

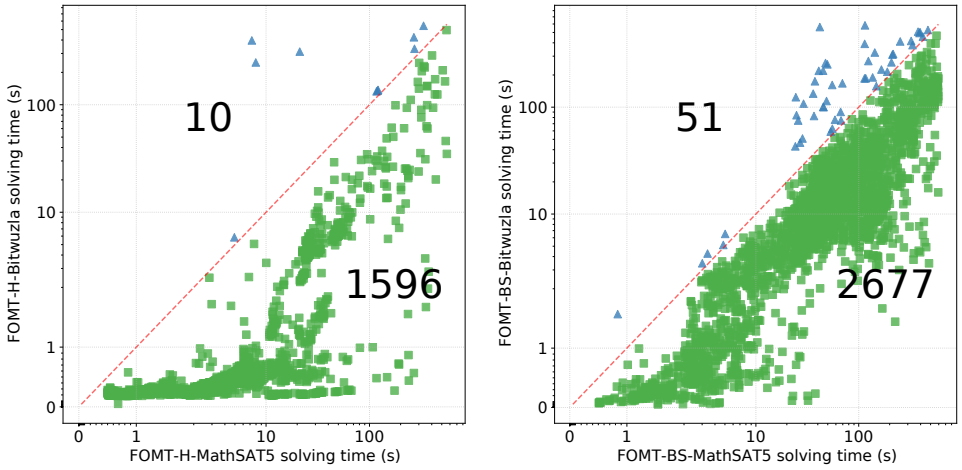
Figure 2 shows the solving time comparison between FOMT-MathSAT5 and FOMT-Bitwuzla on the QF_FP benchmark and the real-world program (RWP) benchmark. The scatter plots are generated in the same manner as in Figure 1, and instances with solving time less than 0.5 s are still excluded. The data are based on 10 independent runs; therefore, a strict comparison is made only when the confidence intervals of the two approaches do not overlap at the 99% confidence level. Numbers indicate instances won (i.e., with shorter solving time) by each approach.

On both the QF_FP and RWP benchmarks, regardless of whether the hybrid strategy or the precise-search-only strategy is used, FOMT-H-M and FOMT-BS-M are faster on fewer instances. In contrast, FOMT-H-B and FOMT-BS-B show clear overall efficiency advantages. In other words, on both benchmarks, the FOMT implementations based on MathSAT5 are generally less efficient than those based on Bitwuzla. This is mainly because Bitwuzla itself performs better than MathSAT5 in floating-point theory solving, indicating that the overall performance of FOMT largely depends on the underlying SMT solver. In particular, on the real-world program benchmark, Bitwuzla significantly outperforms MathSAT5, contributing more substantially to the overall performance improvement of FOMT.



(**a**) QF_FP (**b**) QF_FP

Figure 2. *Cont.*



(**c**) RWP (**d**) RWP

Figure 2. Execution time comparison between FOMT-MathSAT5 and FOMT-Bitwuzla on the two benchmarks.

Answer to **RQ2**: FOMT built on different SMT solvers exhibits significant performance differences. FOMT-Bitwuzla consistently outperforms FOMT-MathSAT5 across all benchmarks, indicating that adopting a stronger SMT solver can further improve both the effectiveness and efficiency of FOMT.

4.6. Ablation Study

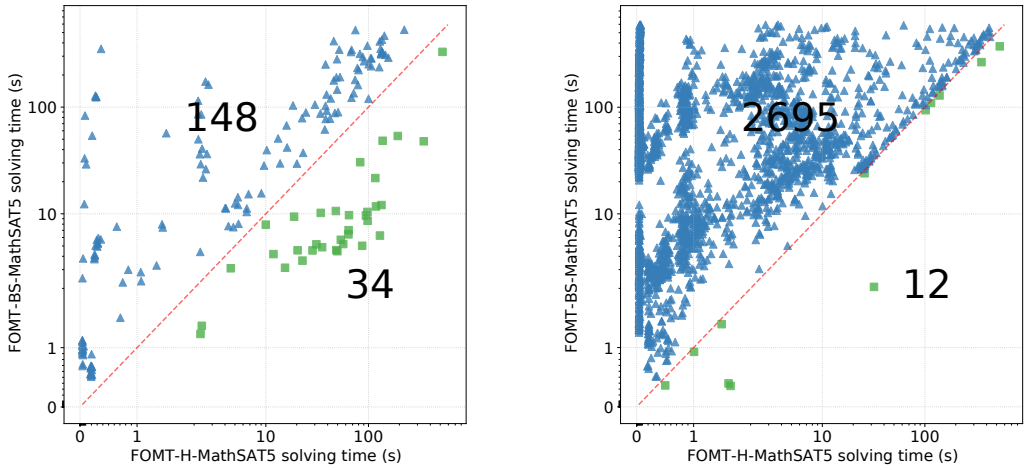
Table 7 compares the solving capability of FOMT-H-MathSAT5 and FOMT-BS-MathSAT5. The table structure is consistent with that of Table 6. The experimental results show that, on the QF_FP benchmark and the real-world program (RWP) benchmark, FOMT-H-M successfully solves 594 and 3374 instances, respectively, whereas FOMT-BS-M solves only 582 and 2863 instances. Further analysis reveals that all instances solved by FOMT-BS-M are also solved by FOMT-H-M, indicating that the hybrid strategy can cover additional solvable instances without sacrificing the capability of precise search. Overall, on both benchmarks, the hybrid strategy of FOMT clearly outperforms the precise-search-only strategy in terms of both the total number of solved instances and the number of uniquely solved instances. This demonstrates that the coarse search phase plays a key role in improving the overall solving capability. In addition, we further conducted statistical significance analysis to evaluate the performance difference between the hybrid strategy and the precise search strategy, as shown in Table 4. Results show that FOMT-H-M significantly outperforms FOMT-BS-M on both benchmarks, confirming the effectiveness of the hybrid strategy.

Table 7. Comparison of the number of solved instances between FOMT-H-M and FOMT-BS-H on the two benchmarks. **Both** denotes the number of instances successfully solved by both methods, **Only FOMT-H-M** denotes the number of instances solved only by FOMT-H-M, **Only FOMT-BS-M** denotes the number of instances solved only by FOMT-BS-M, and **Neither** denotes the number of instances unsolved by both methods.

Benchmark	Both	Only FOMT-H-M	Only FOMT-BS-M	Neither
QF_FP	582 (≈87.26%)	12 (≈1.80%)	0 (≈0.00%)	73 (≈10.94%)
RW	2863 (≈81.96%)	511 (≈14.63%)	0 (≈0.00%)	119 (≈3.41%)

Figure 3 compares the solving-time performance of FOMT-H-MathSAT5 and FOMT-BS-MathSAT5 on the QF_FP benchmark and the real-world program (RWP) benchmark. In

the scatter plots, each point represents the average solving time of an instance (averaged over ten independent runs), and instances with solving time less than 0.5 s are excluded. The horizontal axis denotes the solving time of FOMT-H-M, while the vertical axis denotes that of FOMT-BS-M. As shown in the figure, for most instances, FOMT-H-M achieves lower average solving time than FOMT-BS-M. However, for a small number of instances, FOMT-BS-M performs slightly better. This is mainly because the coarse search phase in the hybrid strategy incurs additional overhead on these instances but fails to produce a useful approximate solution. In contrast, the precise-search-only strategy avoids the optimization search and thus saves computation time.



(**a**) QF_FP (**b**) RWP

Figure 3. Execution time comparison between FOMT-H-M and FOMT-BS-M on the two benchmarks.

Answer to **RQ3**: On both the QF_FP and real-world program benchmarks, the hybrid strategy of FOMT outperforms the precise-search-only strategy in terms of both effectiveness and efficiency. The optimization search phase significantly improves the overall performance of FOMT-H.

5. Discussion

This work is subject to both internal and external threats to validity. Regarding internal threats, the first phase of FOMT-H involves randomness. The approximate solutions obtained in the coarse search phase have a significant impact on the efficiency of locating the optimal solution in the subsequent precise search phase. If the first phase fails to find a useful approximate solution, the second phase degenerates into binary search over the entire floating-point domain, which is computationally expensive and inefficient. To mitigate the impact of randomness on the experimental results, all experiments are repeated ten times, and the average values are reported for statistical analysis, thereby improving the reliability of the conclusions.

Regarding external threats, although the evaluation benchmarks are derived from QF_FP SMT-LIB2 and real-world floating-point programs, they may not cover all types of floating-point constraint optimization problems. Furthermore, OptiMathSAT is used with default parameter settings, while better configurations may exist for specific benchmarks. Similarly, the population size of FPEA is set to 100 based on empirical choice, and better parameter settings may exist. The termination condition of the algorithm is determined by the convergence thresholds λ_1 and λ_2, which are treated as hyperparameters in this work and may require different settings for different problems.

6. Related Work

OMT is an extension of SMT, and SMT has long attracted extensive attention from the research community [23,33,34]. With the rapid development of SMT, OMT has also received increasing attention. Early work by Bjørner and Phan reduced OMT to MaxSAT and implemented a simple OMT solver, v3 [2], on top of Z3 [26]. Nadel and Ryvchin [7] observed that OMT over unsigned bit-vectors can be viewed as lexicographic optimization over the bits of the objective value, from the most significant bit (MSB) to the least significant bit (LSB). Based on this insight, they introduced OMT(UBV). Trentin and Sebastiani [8] later extended this idea to signed bit-vectors, enabling direct solving of OMT(SBV). They also proposed a similar method for floating-point arithmetic, using the notion of attractors to guide the search toward the optimal objective value [9]. To the best of our knowledge, this remains the only existing work specifically targeting OMT(FP), and therefore serves as the baseline in our experiments. OMT techniques have been widely applied to practical problems [35–39].

7. Conclusions

OMT(FP) extends SMT and is an important yet challenging problem in formal methods. OptiMathSAT is the state-of-the-art OMT(FP) solver, but its performance is limited due to frequent SMT solver calls. To address this, we propose a more efficient hybrid approach that first finds an approximate optimal model and then quickly derives the true optimal model. We implemented this method as FOMT-H and evaluated it on QF_FP and real-world program benchmarks. Results show that FOMT-H significantly outperforms OptiMathSAT in both effectiveness and efficiency. Future work will explore extending FOMT-H to other theories of OMT.

Author Contributions: Conceptualization, X.Y.; Methodology, X.Y.; Software, X.Y.; Validation, X.Y.; Formal Analysis, X.Y.; Investigation, X.Y.; Resources, X.Y.; Data Curation, X.Y.; Writing—Original Draft Preparation, X.Y.; Writing—Review & Editing, X.Y., L.S., J.Y., Z.L. and F.Y.; Visualization, X.Y.; Supervision, X.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the National Key R&D Program of China (No. 2022YFB4501903), the National Natural Science Foundation of China (Nos. 62172429, 62032024, U2341212).

Data Availability Statement: The data presented in this study are openly available in GitHub at https://github.com/xuyang-nudt/artifact-xomt.git (accessed on 1 April 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Nieuwenhuis, R.; Oliveras, A. On SAT Modulo Theories and Optimization Problems. In Proceedings of the Theory and Applications of Satisfiability Testing—SAT 2006, 9th International Conference, Seattle, WA, USA, 12–15 August 2006; pp. 156–169. [CrossRef]
2. Bjørner, N.S.; Phan, A. νZ—Maximal Satisfaction with Z3. In Proceedings of the 6th International Symposium on Symbolic Computation in Software Science, SCSS 2014, Gammarth, La Marsa, Tunisia, 7–8 December 2014; pp. 1–9. [CrossRef]
3. Sebastiani, R.; Tomasi, S. Optimization Modulo Theories with Linear Rational Costs. *ACM Trans. Comput. Log.* **2015**, *16*, 12:1–12:43. [CrossRef]
4. Fazekas, K.; Bacchus, F.; Biere, A. Implicit Hitting Set Algorithms for Maximum Satisfiability Modulo Theories. In Proceedings of the Automated Reasoning—9th International Joint Conference, IJCAR 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, 14–17 July 2018; pp. 134–151. [CrossRef]
5. Li, Y.; Albarghouthi, A.; Kincaid, Z.; Gurfinkel, A.; Chechik, M. Symbolic optimization with SMT solvers. In Proceedings of the 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL'14, San Diego, CA, USA, 20–21 January 2014; pp. 607–618. [CrossRef]
6. Yao, P.; Shi, Q.; Huang, H.; Zhang, C. Program analysis via efficient symbolic abstraction. *Proc. ACM Program. Lang.* **2021**, *5*, 1–32. [CrossRef]

7. Nadel, A.; Ryvchin, V. Bit-Vector Optimization. In Proceedings of the Tools and Algorithms for the Construction and Analysis of Systems—22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, 2–8 April 2016; pp. 851–867. [CrossRef]
8. Trentin, P.; Sebastiani, R. Optimization Modulo the Theories of Signed Bit-Vectors and Floating-Point Numbers. *J. Autom. Reason.* **2021**, *65*, 1071–1096. [CrossRef]
9. Trentin, P.; Sebastiani, R. Optimization Modulo the Theory of Floating-Point Numbers. In Proceedings of the Automated Deduction—CADE 27—27th International Conference on Automated Deduction, Natal, Brazil, 27–30 August 2019; pp. 550–567. [CrossRef]
10. Sebastiani, R.; Trentin, P. OptiMathSAT: A Tool for Optimization Modulo Theories. In Proceedings of the Computer Aided Verification—27th International Conference, CAV 2015, San Francisco, CA, USA, 18–24 July 2015; pp. 447–454. [CrossRef]
11. *IEEE Standard 754-1985*; IEEE Standard for Binary Floating-Point Arithmetic. Beuth: Berlin, Germany, 1985.
12. Copi, I.M.; Cohen, C.; McMahon, K. *Introduction to Logic*; Routledge: New York, NY, USA, 2016. [CrossRef]
13. Bradley, A.R.; Manna, Z. *The Calculus of Computation—Decision Procedures with Applications to Verification*; Springer: Berlin/Heidelberg, Germany, 2007. [CrossRef]
14. Rossi, F.; van Beek, P.; Walsh, T. (Eds.) *Handbook of Constraint Programming (Foundations of Artificial Intelligence)*; Elsevier: Amsterdam, The Netherlands, 2006; Volume 2.
15. Norouzi, M.; Fleet, D.J.; Salakhutdinov, R. Hamming Distance Metric Learning. In Proceedings of the 26th International Conference on Neural Information Processing Systems, Lake Tahoe, NV, USA, 3–6 December 2012; pp. 1070–1078.
16. Danielsson, P.E. Euclidean distance mapping. *Comput. Graph. Image Process.* **1980**, *14*, 227–248. [CrossRef]
17. Bäck, T.; Schwefel, H. An Overview of Evolutionary Algorithms for Parameter Optimization. *Evol. Comput.* **1993**, *1*, 1–23. [CrossRef]
18. Yang, X.; Zou, J.; Yang, S.; Zheng, J.; Liu, Y. A Fuzzy Decision Variables Framework for Large-Scale Multiobjective Optimization. *IEEE Trans. Evol. Comput.* **2023**, *27*, 445–459. [CrossRef]
19. Poli, R.; Langdon, W.B. Schema Theory for Genetic Programming with One-Point Crossover and Point Mutation. *Evol. Comput.* **1998**, *6*, 231–252. [CrossRef] [PubMed]
20. Syswerda, G. Uniform Crossover in Genetic Algorithms. In *Proceedings of the 3rd International Conference on Genetic Algorithms, George Mason University, Fairfax, VA, USA, June 1989*; Schaffer, J.D., Ed.; Morgan Kaufmann: San Francisco, CA, USA, 1989; pp. 2–9.
21. Falco, I.D.; Cioppa, A.D.; Tarantino, E. Mutation-based genetic algorithm: Performance evaluation. *Appl. Soft Comput.* **2002**, *1*, 285–299. [CrossRef]
22. Li-min, Z.; Yuan, Z.; Wen-biao, L. The Design of Target Assignment Model Based on the Reverse Mutation Ant Colony Algorithm. *Procedia Eng.* **2012**, *29*, 1554–1558. [CrossRef]
23. Yang, X.; Chen, Z.; Dong, W.; Wang, J. QSF: Multi-objective Optimization Based Efficient Solving for Floating-Point Constraints. *Proc. ACM Softw. Eng.* **2025**, *2*, 511–532. [CrossRef]
24. Cimatti, A.; Griggio, A.; Schaafsma, B.J.; Sebastiani, R. The MathSAT5 SMT Solver. In Proceedings of the Tools and Algorithms for the Construction and Analysis of Systems—19th International Conference, TACAS 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, 16–24 March 2013; pp. 93–107. [CrossRef]
25. Niemetz, A.; Preiner, M. Bitwuzla. In Proceedings of the Computer Aided Verification—35th International Conference, CAV 2023, Paris, France, 17–22 July 2023; pp. 3–17. [CrossRef]
26. de Moura, L.M.; Bjørner, N.S. Z3: An Efficient SMT Solver. In Proceedings of the Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, 29 March–6 April 2008; pp. 337–340. [CrossRef]
27. Lattner, C.; Adve, V.S. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In Proceedings of the 2nd IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2004), San Jose, CA, USA, 20–24 March 2004; pp. 75–88. [CrossRef]
28. Johnson, S.G. The NLopt Nonlinear-Optimization Package. 2007. Available online: https://github.com/stevengj/nlopt (accessed on 1 April 2026).
29. Barrett, C.W.; Stump, A.; Tinelli, C. The SMT-LIB standard: Version 2.0. In Proceedings of the 8th International Workshop on Satisfiability Modulo Theories, Edinburgh, UK, 14–15 July 2010; Volume 13, p. 14.
30. Cadar, C.; Dunbar, D.; Engler, D.R. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, San Diego, CA, USA, 8–10 December 2008; pp. 209–224.
31. Galassi, M.; Davies, J.; Theiler, J.; Gough, B.; Jungman, G.; Alken, P.; Booth, M.; Rossi, F.; Ulerich, R. *GNU Scientific Library*; Network Theory Limited Godalming: Godalming, UK, 2002.
32. Yang, X.; Zhang, G.; Shuai, Z.; Chen, Z.; Wang, J. Symbolic execution of floating-point programs: How far are we? *J. Syst. Softw.* **2025**, *220*, 112242. [CrossRef]

33. Huang, Y.; Niu, Q.; Song, Y. Research on Abstraction-Based Search Space Partitioning and Solving Satisfiability Problems. *Mathematics* **2025**, *13*, 868. [CrossRef]
34. Zhong, Z.; Zhang, Z.; Guan, H.; Li, D. Orax: A Feedback-Driven Framework for Efficiently Solving Satisfiability Modulo Theories and Oracles. *Proc. ACM Program. Lang.* **2025**, *9*, 676–703. [CrossRef]
35. Cimatti, A.; Franzén, A.; Griggio, A.; Sebastiani, R.; Stenico, C. Satisfiability Modulo the Theory of Costs: Foundations and Applications. In Proceedings of the Tools and Algorithms for the Construction and Analysis of Systems, 16th International Conference, TACAS 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, 20–28 March 2010; pp. 99–113. [CrossRef]
36. Larraz, D.; Oliveras, A.; Rodríguez-Carbonell, E.; Rubio, A. Minimal-Model-Guided Approaches to Solving Polynomial Constraints and Extensions. In Proceedings of the Theory and Applications of Satisfiability Testing-SAT 2014-17th International Conference, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, 14–17 July 2014; pp. 333–350. [CrossRef]
37. Kovásznai, G.; Biró, C.; Erdelyi, B. Generating Optimal Scheduling for Wireless Sensor Networks by Using Optimization Modulo Theories Solvers. In Proceedings of the 15th International Workshop on Satisfiability Modulo Theories affiliated with the International Conference on Computer-Aided Verification (CAV 2017), Heidelberg, Germany, 22–23 July 2017; pp. 15–27.
38. Bertolissi, C.; dos Santos, D.R.; Ranise, S. Solving Multi-Objective Workflow Satisfiability Problems with Optimization Modulo Theories Techniques. In Proceedings of the 23nd ACM on Symposium on Access Control Models and Technologies, SACMAT 2018, Indianapolis, IN, USA, 13–15 June 2018; pp. 117–128. [CrossRef]
39. Yin, Y.; Kong, X.; Xia, C.; Xu, C.; Jin, X. Low-cost emergent dynamic scheduling for flexible job shops. *Mathematics* **2022**, *10*, 1873. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.